

AWS Organizations & Service Control Policies (SCPs) — Multi-Account Governance

Executive Summary

Service Control Policies (SCPs) are a powerful AWS Organizations feature that provides centralized permission management across multiple AWS accounts. This comprehensive guide covers the fundamental concepts, practical implementations, best practices, and interview-ready explanations for multi-account governance using AWS Organizations and SCPs.

1. Detailed Explanation of AWS Organizations & SCPs

1.1 What is AWS Organizations?

AWS Organizations is a service that enables you to centrally manage and govern multiple AWS accounts as they grow. It consolidates billing, enforces governance policies, and provides a hierarchical structure for managing your AWS environment.

****Key Components:****

- ****Management Account**** (formerly Root Account): The central account that creates and manages the organization - ****Member Accounts****: AWS accounts that belong to the organization - ****Organizational Units (OUs)****: Container for grouping accounts with shared governance policies - ****Policies****: Rules that are attached to OUs or accounts to control behavior and permissions - ****Roots****: The top-level organizational unit from which all OUs branch

1.2 What are Service Control Policies (SCPs)?

Service Control Policies (SCPs) are JSON-based policies that define the ****maximum permissions**** available to IAM principals (users, roles) within an AWS account. They act as a permission boundary at the organizational level, not replacing but rather limiting IAM policies.

****Critical Principle****: SCPs are a DENY engine. They specify what actions are NOT allowed, creating an upper permission limit.

1.3 How SCPs Work: The Permission Evaluation Logic

When an API call is made in an AWS account with SCPs attached, the permission evaluation follows this hierarchy:

$\text{SCP Permission} \cap \text{IAM Permission} \cap \text{Resource-Based Policy} = \text{Final Permission}$

****Example Scenario:****

User has IAM policy allowing S3 access → SCP denies s3:* → Result: Access DENIED

The intersection means: - If SCP denies an action, it's denied (even if IAM allows it) - If IAM denies an action, it's denied (even if SCP allows it) - Both SCP and IAM must allow for final allow

1.4 Key Characteristics of SCPs

Key SCP Characteristics

Characteristic	Details
Maximum Permissions	Sets upper permission limit, doesn't grant access
Account Coverage	Applies to all IAM users/roles (except root user in management account)
Inheritance	OUs inherit policies from parent OUs
Policy Size	Maximum 5,120 characters per policy
Nesting Depth	Up to 5 levels of nested OUs
Effective Policies	Up to 5 policies per target (OU or account)
Root User Impact	Does NOT affect root user in management account
Regional Scope	Global enforcement (affects all regions)

1.5 SCPs vs IAM Policies vs Permission Boundaries

Comparison: SCPs vs IAM Policies vs Permission Boundaries

Aspect	SCPs	IAM Policies	Permission Boundaries
Scope	Organizational level	Individual user/role	Individual user/role
Purpose	Set maximum permissions	Grant specific access	Define permission ceiling
Management	Central governance	Per-account	Per-account
Effect	DENY-based	ALLOW/DENY	ALLOW/DENY (limit only)

—

2. Practical Examples of Service Control Policies

2.1 Example 1: Restrict AWS Regions

****Use Case**:** Enforce compliance by limiting deployments to approved regions only.

```
"Version": "2012-10-17", "Statement": [ { "Effect": "Deny", "Action": "*", "Resource": "*",
"Condition": "StringNotEquals": "aws:RequestedRegion": ["us-east-1", "eu-west-1", "ap-
southeast-1"] } ]
```

****How it works**:** - Denies ALL actions in regions other than the three specified - Applies to all services and all resources - Effective immediately upon attachment to an OU

****Real-world impact**:** - Users attempting to launch EC2 in us-west-2: Access DENIED - Users working in approved regions: No restrictions

2.2 Example 2: Prevent S3 Bucket Encryption Bypass

****Use Case**:** Ensure all S3 buckets are encrypted by default.

```
"Version": "2012-10-17", "Statement": [ { "Effect": "Deny", "Action": "s3:CreateBucket",
"Resource": "*", "Condition": "StringNotEquals": "s3:x-amz-server-side-encryption":
"AES256" }, { "Effect": "Deny", "Action": "s3:PutObject", "Resource": "*", "Condition":
"StringNotEquals": "s3:x-amz-server-side-encryption": ["AES256", "aws:kms"] } ]
```

****How it works**:** - Blocks bucket creation without AES256 encryption - Blocks object uploads without proper encryption - Forces encryption at the infrastructure level

****Benefits**:** - Prevents accidental unencrypted data storage - Eliminates compliance violations - Reduces security audit findings

2.3 Example 3: Restrict Dangerous IAM Actions for Non-Admin Roles

****Use Case**:** Prevent accidental or malicious modifications to IAM policy structure.

```
"Version": "2012-10-17", "Statement": [ { "Effect": "Deny", "Action": [ "iam:DeletePolicy",
"iam:DeleteUser", "iam:DeleteRole", "iam:DeleteAccessKey", "iam:CreateAccessKey",
"iam:AttachUserPolicy", "iam:AttachRolePolicy" ], "Resource": "*", "Condition":
"StringNotEquals": "aws:PrincipalOrgID": "*" } ]
```

****How it works**:** - Blocks high-risk IAM modifications - Only security administrators (with explicit Allow) can perform these actions - Creates audit trail through CloudTrail

2.4 Example 4: Prevent Root User Credential Management

****Use Case**:** Reduce risk of root account compromise.

```
"Version": "2012-10-17", "Statement": [ { "Sid": "DenyRootUserActions", "Effect": "Deny",
"Action": [ "iam:CreateAccessKey", "iam:CreateLoginProfile", "iam:DeleteUser",
"iam:DeleteAccessKey" ], "Resource": "arn:aws:iam::*:root", "Condition": "Bool":
"aws:PrincipalIsAWSRoot": true } ]
```

****How it works:**** - Prevents root user from creating access keys - Prevents root user creation modifications - Root user can only be managed through specific approval workflows

2.5 Example 5: Enforce EC2 Instance Types and IMDSv2

****Use Case**:** Prevent expensive instance types and enforce security metadata service.

```
"Version": "2012-10-17", "Statement": [ "Sid": "DenyExpensiveInstanceTypes", "Effect": "Deny", "Action": "ec2:RunInstances", "Resource": "arn:aws:ec2:*:*:instance/*", "Condition": "StringLike": "ec2:InstanceType": [ "*.8xlarge", "*.9xlarge", "*.10xlarge", "*.12xlarge" ] , "Sid": "RequireIMDSv2", "Effect": "Deny", "Action": "ec2:RunInstances", "Resource": "arn:aws:ec2:*:*:instance/*", "Condition": "StringNotEquals": "ec2:MetadataHttpTokens": "required" ]
```

****How it works:**** - Blocks xlarge+ instance types to control costs - Forces IMDSv2 for enhanced security - Prevents credential exposure through metadata service exploitation

3. SCP Implementation Strategy: Multi-Account Governance Model

3.1 Organizational Structure Example

Typical AWS Organizations Structure with OUs and SCPs

Typical AWS Organizations Structure with OUs and SCPs

****Hierarchy:**** - ****Root****: Organization root container - ****Management OU****: Central billing and governance account - ****Security OU****: Centralized logging, CloudTrail, Config, GuardDuty - ****Production OU****: Mission-critical workloads with strictest controls - ****Development OU****: Development and testing with moderate controls - ****Sandbox OU****: Experimentation with minimal controls (except security baseline)

3.2 SCP Application Strategy by OU Level

SCP Application Strategy by OU

OU Type	SCPs Applied	Purpose
Security OU	Minimal (foundational only)	Specialized operations, high trust
Production OU	Maximum enforcement	Prevent accidental changes, enforce compliance
Development OU	Moderate restrictions	Balance agility with governance
Sandbox OU	Minimal (cost controls)	Enable experimentation and learning

3.3 Foundational SCP for All Accounts

Applied to Root level (affects all accounts):

- Restrict to approved regions
- Deny root user credential management
- Enforce encryption for data at rest
- Require MFA for sensitive actions
- Prevent account removal from organization

3.4 Progressive SCP Implementation Phases

1. **Phase 1 (Week 1-2):** Test in Development

- Create foundational SCPs
- Test in dev account first
- Monitor CloudTrail for unexpected denials

2. **Phase 2 (Week 3-4):** Deploy to Sandbox

- Apply policies to Sandbox OU
- Gather feedback from users
- Adjust based on real-world usage

3. **Phase 3 (Week 5-6):** Production Preparation

- Fine-tune policies based on lessons learned
- Create exceptions/exemptions framework
- Brief stakeholders on changes

4. **Phase 4 (Week 7-8):** Production Rollout

- Apply to Production OU
- Monitor closely for legitimate denials
- Have rollback plan ready

—

4. Top 5 Interview Questions on AWS Organizations & SCPs

Question 1: Multi-Account Architecture Design

****Q:** Your organization is migrating 50 workloads from a single AWS account to a multi-account strategy using AWS Organizations. Design a comprehensive governance model using SCPs that addresses security, cost optimization, and operational efficiency.[1]******

****Model Answer:****

I would design a three-tier governance model:

****Account Structure:****

- **Management Account:** Centralized billing, organization management, no production workloads
- **Security Account:** CloudTrail aggregation, Config rules, GuardDuty, IAM policies, audit logs
- **Shared Services Account:** VPN endpoints, repositories, secrets management
- **Production Account Group:** Segregated by team/application with production workloads
- **Development Account Group:** Per-team development and testing environments
- **Sandbox Account:** Unrestricted experimentation for learning

****SCP Strategy:****

****Foundation Layer**** (Applied to Root): "Restrict to approved regions": "Deny non-approved regions", "Prevent account removal": "Deny LeaveOrganization", "Deny root user access key creation": "Blocks root compromise vector"

****Security Layer**** (Applied to all OUs): "Enforce encryption for new S3 buckets": "AES256 or AWS KMS", "Require MFA for sensitive IAM actions": "DeleteUser, CreateAccessKey", "Prevent public S3 bucket creation": "Block PublicRead, PublicReadWrite ACLs"

****Cost Control Layer**** (Applied to Development/Sandbox): "Deny expensive instance types": "Block *.8xlarge and larger", "Require resource tagging": "Tag enforcement for chargeback", "Auto-terminate untagged resources": "After 30 days"

****Implementation Benefits:****

- Centralized billing with per-team cost visibility
- Blast radius containment (breach in one account doesn't affect others)
- Enforced compliance at infrastructure level
- Clear permission boundaries between environments
- Audit trail through CloudTrail in Security Account

****What sets this answer apart:**** - Addresses business outcomes (cost, security, operations)
- Provides concrete policy examples - Explains blast radius containment - Shows progressive implementation approach

Question 2: SCP Permission Evaluation and Troubleshooting

****Q:** A developer in your Production account has IAM permissions to launch t2.micro EC2 instances, but their request is being denied. The SCP attached to the Production OU allows all EC2 actions except *.8xlarge instance types. Explain why the request is failing and how you would troubleshoot it.[2]******

****Model Answer:****

****Analysis:****

The developer's request is failing because of the SCP and IAM permission evaluation logic:

$$\text{SCP} \cap \text{IAM Policy} \cap \text{Resource Policy} = \text{Final Permission}$$

Since the SCP only denies *.8xlarge instances, and the IAM policy allows t2.micro, the intersection should allow the action. Therefore, the issue is not the SCP.

****Troubleshooting Steps:****

1. **Verify IAM Policy Attachment:**

- Check if the IAM policy is directly attached to user/role
- Verify policy syntax and Resource/Action fields
- Check for inherited Deny statements through group membership

2. **Check SCPs on Account:**

- Verify SCP attachment path (Root → OU → Account)
- Check for implicit denies or Deny statements on EC2:RunInstances
- Use Policy Simulator to test SCP evaluation

3. **Verify Resource-Based Policies:**

- Check VPC/Subnet resource policies
- Check security group rules
- Verify AMI permissions

4. **Use AWS Policy Simulator:**

- Service: EC2

- Action: RunInstances
- Result shows which policy is denying (IAM/SCP/Resource)

5. **Check CloudTrail:**

- Look for AccessDenied events
- Error message reveals which policy denied the action
- Example: "User: arn:aws:iam::123456789012:user/dev is not authorized to perform: ec2:RunInstances"

****Most Likely Causes:****

- Implicit Deny in IAM policy (no explicit Allow for ec2:RunInstances)
- Additional SCP denying EC2 actions exists
- User lacks RunInstances permission on specific resources (security groups, subnets, etc.)
- Service-linked role permissions are missing

****What sets this answer apart:**** - Demonstrates understanding of permission evaluation logic - Shows structured troubleshooting methodology - References AWS tools (Policy Simulator, CloudTrail) - Addresses likely causes not just symptoms

—

Question 3: SCP Design for Compliance and Risk Management

****Q:** Your organization handles healthcare data (HIPAA compliance required). Design an SCP strategy that ensures data never leaves the organization, enforces encryption, and prevents unauthorized deletion of logs.[3]

****Model Answer:****

****Compliance Requirements:****

- Data residency: Data must remain in approved regions only
- Data encryption: All data must be encrypted in transit and at rest
- Audit trail: Complete audit logs for 7 years minimum
- Access control: No public access to sensitive resources
- Delete protection: Prevent accidental/malicious data deletion

****SCP Implementation:****

****SCP 1: Data Residency Enforcement****


```
"Version": "2012-10-17", "Statement": [ "Sid": "RestrictToApprovedRegions", "Effect":
"Deny", "Action": "*", "Resource": "*", "Condition": "StringNotEquals":
"aws:RequestedRegion": ["us-east-1", "us-west-2"] ]
```

****SCP 2: Encryption Enforcement****

```
"Version": "2012-10-17", "Statement": [ "Sid": "RequireS3Encryption", "Effect": "Deny",
"Action": ["s3:PutObject"], "Resource": "*", "Condition": "StringNotEquals": "s3:x-amz-
server-side-encryption": ["aws:kms"] , "Sid": "RequireEBSEncryption", "Effect": "Deny",
"Action": ["ec2:RunInstances"], "Resource": "arn:aws:ec2:*:*:volume/*", "Condition":
"Bool": "ec2:Encrypted": "false" , "Sid": "RequireRDSEncryption", "Effect": "Deny",
"Action": ["rds:CreateDBInstance"], "Resource": "*", "Condition": "Bool":
"rds:StorageEncrypted": "false" ]
```

****SCP 3: Prevent Deletion of Audit Logs****

```
"Version": "2012-10-17", "Statement": [ "Sid": "ProtectCloudTrailLogs", "Effect": "Deny",
"Action": [ "s3:DeleteObject", "s3:DeleteBucket", "logs:DeleteLogGroup",
"logs:DeleteRetentionPolicy" ], "Resource": [ "arn:aws:s3:::audit-logs-*",
"arn:aws:logs:*:*:log-group:/aws/cloudtrail/*" ], "Sid": "PreventCloudTrailDisable",
"Effect": "Deny", "Action": [ "cloudtrail:StopLogging", "cloudtrail:DeleteTrail",
"cloudtrail:UpdateTrail" ], "Resource": "*" ]
```

****SCP 4: Prevent Public Access****

```
"Version": "2012-10-17", "Statement": [ "Sid": "PreventPublicS3", "Effect": "Deny",
"Action": [ "s3:PutAccountPublicAccessBlock", "s3:PutBucketPublicAccessBlock" ],
"Resource": "*", "Condition": "Bool": "s3:BlockPublicAcls": "false" ]
```

****Monitoring and Compliance:****

- CloudTrail logs all API calls (audit trail for 7 years in S3 Glacier)
- AWS Config tracks encryption status and public access
- CloudWatch alarms on SCP denials (indicates policy violations)
- Quarterly audit reports using AWS Security Hub

****What sets this answer apart:**** - Addresses specific compliance requirements (HIPAA) -
Provides concrete, production-ready policies - Shows defense-in-depth (multiple layers) -
Includes monitoring strategy - Explains data residency requirements

—

Question 4: SCP Exceptions and Exemption Frameworks

****Q:** You have strict SCPs preventing deletion of production resources. However, a legitimate operational need requires quarterly backups to be deleted after 30 days. How would you architect an exception framework that maintains security while allowing necessary operations?[4]**

****Model Answer:****

****Problem:**** SCPs that deny DeleteObject for production resources prevent legitimate backup cleanup, creating cost waste and storage bloat.

****Solution Architecture:****

****Option 1: Dedicated Service Account (Recommended)****

1. Create Backup Service Account:

- Separate AWS account for backup management
- Apply relaxed SCPs only to this account
- All backup cleanup operations routed through this account

2. SCP for Backup Account:

- Allow s3:DeleteObject for backup S3 buckets only
- Deny all other delete operations (production resources safe)
- Maintain encryption and region restrictions

3. Cross-Account Role:

- Production account creates IAM role for backup account
- Role limited to s3:DeleteObject on backup-* buckets only
- Trust relationship restricted to backup account principal

4. Audit Trail:

- CloudTrail shows which backups were deleted by whom
- All deletion actions go through explicit role assumption

```
"Version": "2012-10-17", "Statement": [ "Sid": "AllowBackupDeletion", "Effect": "Allow",  
"Action": ["s3:DeleteObject"], "Resource": "arn:aws:s3:::backup-*/*", "Condition":  
"StringEquals": "aws:PrincipalArn":  
"arn:aws:iam::BACKUP-ACCOUNT:role/BackupCleanupRole" ]
```

****Option 2: Attribute-Based Access Control (ABAC)****

1. Tag-Based Permissions:

- Tag resources with backup-lifecycle=temporary
- SCP allows deletion ONLY on resources with this tag
- Permanent production resources have backup-lifecycle=permanent

2. SCP Implementation:

```
"Version": "2012-10-17", "Statement": [ "Sid": "DenyDeleteForPermanentResources",
"Effect": "Deny", "Action": ["s3:DeleteObject", "ec2:TerminateInstances"], "Resource": "*",
"Condition": "StringNotEquals": "aws:PrincipalTag/backup-lifecycle": "temporary" ]
```

****Option 3: Time-Based Exceptions (Advanced)****

Use AWS Lambda + EventBridge for scheduled cleanup:

- EventBridge triggers Lambda on schedule (daily at 2 AM UTC)
- Lambda assumes service role with explicit backup deletion permissions
- Cleanup logic: find backups older than 30 days, delete them
- CloudTrail logs all deletions by Lambda role
- SNS notifications on completion

****Best Practices for Exception Framework:****

- **Principle of Least Privilege:** Exceptions limited to specific resources/actions
- **Audit Trail:** All exceptions logged and traceable
- **Expiration:** Temporary exceptions auto-expire (quarterly review)
- **Approval Process:** Change control board approves new exceptions
- **Monitoring:** CloudWatch alerts on exception usage

****What sets this answer apart:**** - Provides multiple solution options - Shows architectural thinking (separate account vs ABAC vs automation) - Maintains security while enabling operations - Demonstrates audit trail importance - Explains practical implementation details

—

Question 5: SCP Troubleshooting and Real-World Failures

****Q:** You deployed an SCP last night that was supposed to prevent expensive instance types. This morning, developers report they cannot launch ANY EC2 instances in the Development account. The SCP policy uses a Condition block but is denying all instance types. What's likely wrong, and how would you fix it?[5]

****Model Answer:****

****Most Likely Issue:** Incorrect Condition Logic (Logical AND vs OR)

The SCP likely has a logic error in how conditions are evaluated.

****Common Mistake:****

```
"Effect": "Deny", "Action": "ec2:RunInstances", "Resource": "arn:aws:ec2:*:*:instance/*",  
"Condition": "StringLike": "ec2:InstanceType": [ "*.8xlarge", "*.9xlarge" ]
```

Problem: AWS evaluates conditions with logical AND. If ANY condition is false, the entire Deny statement fails. Multiple values in StringLike use OR logic within the key, but if the key doesn't match, the entire policy doesn't apply.

****The Real Problem:**** The SCP is likely matching too broadly or condition key is wrong.

****Troubleshooting Steps:****

1. **Enable CloudTrail:**

- Query CloudTrail for recent EC2 API calls
- Look for errorCode: "UnauthorizedOperation"
- Error message reveals which policy/condition is denying

2. **Use Policy Simulator:**

- Service: EC2
- Action: RunInstances
- Result shows exact policy causing denial

3. **Common Issues:**

- Condition key typo (ec2:InstanceType vs wrong key name)
- Wrong Resource ARN pattern (resource doesn't match actual API call)
- Accidental global Deny on all EC2 actions
- Multiple SCPs with conflicting deny statements

****Correct Implementation:****

```
"Version": "2012-10-17", "Statement": [ "Sid": "DenyExpensiveInstanceTypesCorrect",  
"Effect": "Deny", "Action": "ec2:RunInstances", "Resource": "arn:aws:ec2:*:*:instance/*",  
"Condition": "StringLike": "ec2:InstanceType": [ "*.8xlarge", "*.9xlarge", "*.10xlarge" ] ]
```

****If still failing, check:****

1. **Multiple SCPs:**

- OU might have multiple SCPs attached
- One of them might have unintended Deny
- Review all SCPs attached to OU and account

2. **OU Inheritance:**

- Development OU inherits from Parent OU
- Parent OU might have restrictive SCP
- Check entire OU chain: Root → Parent → Dev OU

3. **Implicit Deny:**

- SCP doesn't have a matching Allow statement
- SCPs only set maximum permissions, don't grant
- IAM policy in the account must have Allow for RunInstances

****Immediate Remediation:****

1. Detach problematic SCP from Development OU 2. Developers regain access immediately
3. Review SCP in development environment 4. Use Policy Simulator to validate logic 5. Reattach after testing and validation 6. Document lesson learned

****Prevention Strategy:****

- **Test Before Production:** Apply SCPs to test OU first
- **Gradual Rollout:** 24-hour observation period before expanding
- **Monitoring:** CloudWatch alarms on AccessDenied counts
- **Runbook:** Have rollback procedures documented
- **Peer Review:** SCPs reviewed by 2 people before deployment

****What sets this answer apart:**** - Identifies the likely root cause (condition logic) - Provides structured troubleshooting methodology - Uses AWS tools (CloudTrail, Policy Simulator) - Addresses multi-SCP scenarios - Includes remediation and prevention strategies - Shows real operational experience

5. Best Practices Summary

- **Start with minimal SCPs:** Begin with foundational policies (region restrictions, root user protection)
- **Test in development first:** Use test OUs before production rollout
- **Use descriptive Sids:** Each statement needs clear, self-documenting identifiers
- **Monitor with CloudTrail:** Maintain complete audit trail of SCP denials
- **Gradual implementation:** Roll out over 4-8 weeks, gathering feedback

- **Document exceptions:** Maintain clear record of who can do what and why
- **Review quarterly:** SCPs should evolve with organizational needs
- **Use Policy Simulator:** Always validate SCP logic before production deployment
- **Combine with IAM roles:** SCPs work best with centralized IAM role management
- **Automate enforcement:** Use AWS Config rules to detect SCP violations

References

- [1] AWS Organizations Documentation. (2024). Service control policies (SCPs).
https://docs.aws.amazon.com/organizations/latest/userguide/orgs_manage_policies_scps.html
- [2] AWS Identity and Access Management User Guide. (2024). Permission evaluation logic.
https://docs.aws.amazon.com/IAM/latest/UserGuide/reference_policies_evaluation-logic.html
- [3] AWS Compliance Center. (2024). HIPAA compliance on AWS.
<https://docs.aws.amazon.com/compliance/latest/userguide/hipaa.html>
- [4] AWS Organizations Best Practices. (2024). Multi-account strategy.
<https://docs.aws.amazon.com/organizations/latest/userguide/>
- [5] AWS CloudTrail User Guide. (2024). Logging AWS Organizations API calls.
<https://docs.aws.amazon.com/cloudtrail/latest/userguide/>